Entender y practicar las Colecciones en Java

Este documento tiene el objetivo de ayudaros a repasar las colecciones en java, ya que varios alumnos de la clase decían que no las dieron en primero, en el módulo de programación.

En primer lugar os incluyo un resumen de los conceptos principales y las diferencias de las Colecciones.

Después, os planteo un ejercicio que os obligará a practicar con los métodos básicos de las Colecciones en java. Me refiero a métodos básicos como son: add, get, remove, containsKey, size, iterator/entrySet. También, este ejercicio está pensado para entender las diferencias fundamentales de cómo cada colección maneja el **orden** y la **unicidad**.

Además de estudiar estos mini-aputes y realizar el ejercicio, debéis repasar y practicar creando funcionalidades que utilicen colecciones y que os sirvan para vuestro proyecto. Espero que lo hagáis y me lo mostréis también.

1. Conceptos Fundamentales de Colecciones en Java

Manejar bien las **Colecciones** en Java es muy importante para poder desarrollar aplicaciones avanzadas. Son la base para manejar grupos de objetos de forma eficiente.

Estos apuntes se enfocan en **ArrayList** (que es una **List**), **HashSet** (que es un **Set**) y **HashMap** (que es un **Map**). Primero se explica cada concepto y luego se propone el ejercicio práctico de refuerzo de ese concepto.

Las Colecciones (java.util.*) son *estructuras de datos* que nos permiten almacenar, recuperar y manipular objetos.

a) List (Implementada con ArrayList)

- **Concepto:** Es una colección **ordenada** que permite **elementos duplicados**. Piensa en ella como un **array dinámico** cuyo tamaño puede crecer o reducirse. El orden de inserción se mantiene.
- **Uso principal:** Cuando el **orden** de los elementos y la posibilidad de tener **duplicados** son importantes.
- **Clave:** Acceso por **índice** (posición).
- **Ejemplo:** Una lista de nombres de clientes *donde el mismo nombre puede aparecer varias veces*.

b) Set (Implementado con HashSet)

- **Concepto:** Es una colección que **no permite elementos duplicados**. El orden de los elementos **no está garantizado.** La principal diferencia entre Set y HashSet es que Set es una interfaz que define el comportamiento de una colección de elementos únicos, mientras que HashSet es una implementación específica de esa interfaz que utiliza tablas hash para almacenar los elementos de forma no ordenada y con alta eficiencia para las operaciones de búsqueda y adición. Por lo tanto, un Set no se puede instanciar directamente, pero sí se puede crear un HashSet como una de sus implementaciones.
- **Uso principal:** Para asegurar que cada elemento es **único**.
- **Clave: Unicidad**. Para que funcione correctamente, la clase del objeto debe implementar correctamente hashCode() y equals().
- **Ejemplo:** Una lista de números de identificación de productos **únicos**.

c) Map (Implementado con HashMap)

- **Concepto:** Es una colección que almacena pares de **clave-valor** (Key-Value). Cada **clave** es **única** y se utiliza para recuperar su **valor** asociado. Ni el orden ni la duplicidad se garantizan (solo las claves deben ser únicas, los valores pueden duplicarse).
- **Uso principal:** Para buscar datos rápidamente usando una clave específica.
- **Clave:** Almacenamiento y recuperación por **Clave Única**.
- **Ejemplo:** Un diccionario donde la palabra es la *clave* y la definición es el *valor*. O, un mapa de códigos de país (ES, MX, AR) a nombres de país (España, Méjico, Argentina).

2. Ejercicio Práctico: Gestión de un Inventario

Os propongo un ejercicio de simulación de un **Sistema de Inventario Básico** que necesita usar las tres estructuras de datos.

Escenario:

Una tienda necesita gestionar su inventario de productos, para lo que vas a desarrollar un servicio de inventario.

Cada producto del inventario tiene un **Código**(String), un **Nombre**(String) y un número de **Exitencias**(Integer).

Instrucciones del Ejercicio:

Se deben crear dos ficheros java: una clase que se llame InventarioServicio y una App para el main. La clase InventarioServicio tendrá tres métodos. Cada método será invocado en App para implementar el comportamiento de las tres actividades siguientes:

Actividad 1: procesarListaCompras (Lista de Compras del Día de tipo ArrayList)

Se pide:

1. Crear un ArrayList de Strings que se llame listaDemo.
2. Invocar al método procesarListaCompras que:
    - Añadirá los siguientes ítems *en este orden*: "Monitor", "Teclado", "Monitor", "Ratón", "Cable HDMI".
    - Pintará la lista completa para demostrar que **mantiene el orden** y permite **duplicados**.
    - **Quitará el elemento** en el índice 2 (que debería ser el segundo "Monitor").
3. Pintar la lista resultante.

Entender y practicar las Colecciones en Java

Actividad 2: obtenerCodigosUnicos (Registro de Productos Únicos de tipo HashSet)

Se pide:

1. Crear un Set de String que se llame codigosDemo.
2. Invocar al método obtenerCodigosUnicos que:
    - Añadirá los siguientes códigos: "A101", "B202", "A101", "C303", "B202".
    - Pintará el Set completo para demostrar que **solo almacena los elementos únicos** y que el **orden no está garantizado**.
    - Comprobará **si el código** "C303" **existe** en el Set.
3. Pintar todos los códigos resultantes.

Actividad 3: actualizarInventario (Inventario Principal de tipo HashMap)

Se pide:

1. Crear un HashMap que se llame inventarioDemo2, donde la **clave** será el **Código** del producto (String) y el **valor** será el **Nombre** del producto (String).
2. Invocar al método actualizarInventario que:
    - Añadirá los siguientes pares Clave-Valor:
        - "A101": "Laptop"
        - "B202": "Impresora"
        - "C303": "Router"
    - Pintará el Set completo para demostrar su contenido actual.
    - Añadirá el par "A101": "Laptop Pro" (esto **sustituirá** el valor anterior de la clave "A101").
    - Pintará el producto "A101" para **demostrar su contenido actual** es "Laptop Pro".
    - Pintará el producto "B202" para **obtener y mostrar** el nombre del producto asociado.
3. **Iterar** sobre el Map (mostrando tanto la clave como el valor de cada par) y demostrar su contenido final. Se puede usar un for y entrySet().